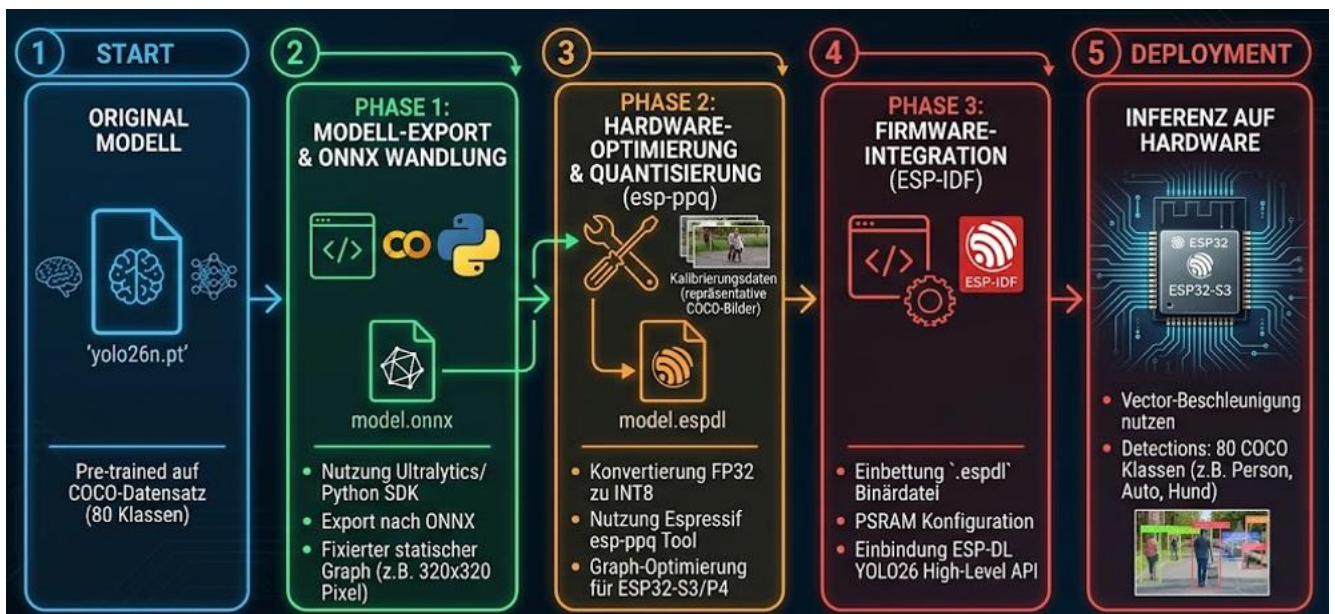


	IT-Technik cyber-physische Systeme		Yolo26 Modell in ESP-DL konvertieren	
	Nachname, Vorname:		Klasse: IF11	Datum:

Wenn man das Yolo26 Modell mit ESP-DL ausführen lassen möchte, muss quantisiert und in das passende Format konvertiert werden. Die Hauptarbeit übernimmt die Funktion `espdL_quantize_onnx` aus dem `esp-ppq` Package von Espressif.

Ein Colab-Notizbuch (Jupyter), das die Konvertierung durchführt findet man unter:
<https://github.com/docblock/06ESP32p4webcamH264Yolo26/blob/main/YOLO26inDL.ipynb>

Das vortrainierte Yolo26-Modell von Ultralytics liegt in dem proprietären `pt`-Format vor, dieses muss zunächst ins ONNX-Format konvertiert werden und wird dann in einem Schritt quantisiert und für den `esp32` optimiert (`espdL_quantize_onnx`). Das sind die Schritte 1 bis 3 aus der Grafik.



Der ESP32-P4 ist ein extrem leistungsfähiger, Dual-Core RISC-V-Mikrocontroller (400 MHz). Er verfügt über Vektorbefehlssätze (SIMD) und spezielle Beschleuniger für künstliche Intelligenz. Dennoch unterscheidet er sich fundamental von einem PC oder Server. Ohne Anpassung scheitert der Einsatz von Standard-KI-Modellen aus drei Gründen:

1. **Datentyp-Inkompatibilität (FP32 vs. INT8)**: Standardmodelle (aus PyTorch oder TensorFlow) rechnen mit 32-Bit-Fließkommazahlen (Float32). Mikrocontroller besitzen oft keine oder nur langsame FP32-Vektoreinheiten. Der ESP32-P4 hingegen besitzt 128-Bit-Vektorregister. Wenn das Modell auf 8-Bit-Integers (Int8) komprimiert wird, passen exakt 16 Parameter gleichzeitig in ein einziges Register. Die Berechnungen laufen dadurch massiv parallelisiert ab.
2. **Speicherplatz (RAM & Flash)**: Ein neuronales Netz im FP32-Format belegt viermal so viel Speicher wie ein INT8-Modell. Selbst mit dem großen PSRAM des ESP32-P4 zählt jedes Kilobyte, um Latenzen zu minimieren.
3. **Graphen-Optimierung**: Unnötige Operationen, die der ESP32-P4 nicht in Hardware beschleunigen kann, werden verschmolzen oder umstrukturiert.
4. **Formatstruktur (Protobuf vs. FlatBuffers)**: Das universelle ONNX-Format nutzt Protobuf zur Serialisierung. Das Parsen davon benötigt auf einem Mikrocontroller zu viel CPU-Leistung und RAM. Espressif benötigt ein eigenes Binärformat (.espdL), das auf FlatBuffers basiert. Dieses erlaubt ein direktes Auslesen aus dem Speicher ohne vorheriges Entpacken (Zero-Copy Deserialization).

Da man so gut wie jedes KI-Modell ins ONNX-Format überführen kann, ist es nachvollziehbar, dass das Espressif-Skript nicht jedes Modell ohne weiteres erfolgreich konvertieren kann. Gerade beim

	IT-Technik cyber-physische Systeme		Yolo26 Modell in ESP-DL konvertieren	
	Nachname, Vorname:		Klasse: IF11	Datum:

YOLO26-Modell gibt es mehrere Fehlerquellen, die abgefangen werden müssen. Diese sind in dem colab-Notizbuch ausführlich beschrieben.

Insbesondere die Ausgabe des Modells muss angepasst werden. Normalerweise bekomme ich als Ergebnis des Detektionskopfes im Modell direkt die Boundingboxen der erkannten Objekte. Die Operationen, die in diesem Detektionskopf verwendet werden, können nicht vom KI-Befehlssatz des ESP32 beschleunigt werden und müssen entfernt werden. Operationen wie Reshape, Concat, Softmax (für die Distribution Focal Loss) oder gar das rechenintensive NMS (Non-Maximum Suppression) kosten auf einem Mikrocontroller wertvolle Millisekunden oder erzeugen Quantisierungsfehler.

Anstelle einer sauberen Liste von Bounding Boxes liefert das Modell nun getrennte, rohe Feature-Maps (Ausgangstensoren) für die verschiedenen Erkennungsstufen (Strides). Typischerweise hat YOLO drei Stufen (Stride 8, 16 und 32), die für kleine, mittlere und große Objekte zuständig sind. Für jede dieser Stufen erhält man zwei völlig getrennte Ausgabe-Zweige (Branches):

1. Der Box-Zweig (Regressions-Output): Liefert einen Tensor, der für jede Rasterzelle (Grid Cell) genau 4 Kanäle enthält. Da YOLO26 auf RegMax=1 (direkte Regression) setzt, statt wie ältere YOLO-Modelle DFL mit 16 Kanälchen pro Koordinate zu nutzen, sind dies direkt die 4 Rohwerte für die Geometrie. Meistens stehen diese für l, t, r, b (left, top, right, bottom), also die Abstände von der aktuellen Rasterzelle zu den vier Rändern der Box.
2. Der Score-Zweig (Klassifikations-Output): Liefert einen separaten Tensor, der die rohen Vorhersagewerte für deine Objektklassen enthält (\$C\$ Kanäle für \$C\$ Klassen) – ebenfalls aufgeteilt auf jede Rasterzelle.

Warum getrennt? Box-Werte und Klassifikations-Scores haben völlig unterschiedliche Wertebereiche. Würde man sie im Modell zusammenführen (Concatenate), würde die INT8/INT16-Quantisierung von espdl_quantize_onnx zu massiven Präzisionsverlusten führen.

Die Berechnung der Bounding Boxen erfolgt dann in Software auf dem ESP32. Hier profitiert man von der Architektur von YOLO26. Während man bei älteren Modellen mittels Non-Maximum Suppression (NMS) mühsam überlappende Boxen berechnen und aussortieren musste (IoU-Überlappung), nutzt YOLO26 einen sogenannten One-to-One Head.

Das Modell ist so trainiert, dass es pro Objekt standardmäßig nur noch eine einzige Gitterzelle triggert. Das bedeutet für uns: Man braucht kein NMS mehr. Man sortiert die gefilterten Boxen einfach nach dem höchsten Score und nimmt die besten Ergebnisse (Top-K, z. B. die Top-10). Genau dieser Wegfall von NMS sorgt dafür, dass YOLO26 auf dem ESP32-P4 bis zu 30 % schneller läuft als ältere Architekturen.